

Real Time Liveness Detection and Face Recognition with OpenCV and Deep Learning

Sourav Mandol, Suman Mia, Sk. Md. Masudul Ahsan

Department of Computer Science and Engineering

Khulna University of Engineering & Technology

Khulna-9203, Bangladesh

souravmandol1228@gmail.com, mia1607045@stud.kuet.ac.bd, smahsan@cse.kuet.ac.bd

Abstract— Face has been used to identify people since the beginning of time. Face recognition and liveness detection in real time is a rapidly growing research area. In this paper, a real-time GUI (Graphical User Interface) based system with the combination of facial liveness detection and face recognition has been implemented. This proposed system is capable of detecting spoofing or fakeness. After detecting a real person, the system can determine whether or not the individual is known. This paper covers building a camera-based system using open-source software which accuracy was found 99.33 percentages. This proposed system can also rename or delete a known person without requiring retraining.

Keywords— Liveness detection, Spoofing, Face recognition, OpenCV, Deep Learning, TensorFlow, Keras, dlib, HOG, GUI.

I. INTRODUCTION

With the passage of time, reliance of men on technology to meet their daily basic needs has grown. As a result, Artificial Intelligence, Computer Vision, biometrics, and security systems are rapidly improving. Currently, these fields have developed into a strong and reliable research field.

Face detection and recognition has become a very popular study area in the last 30 years. Face recognition technology has evolved rapidly during the previous 50 years [1]. Face recognition is a technique in which a system uses several types of models to detect and recognize people from a dataset. The technology uses the visage or any other face-related attributes to identify persons. According to a survey [2], facial recognition technology was the second most widely utilized biometric technology roughly ten years ago. Face recognition technology is still improving with the addition of liveness detection. The outcome of the face recognition system is determined by the dataset, algorithms, APIs, and libraries used. It is tough to say which combination will work best in every situation.

This research aims to implement a highly accurate and capable of being executed in real-time face recognition system on desktop that can verify the spoofing before recognizing the person using the webcam. OpenCV, Python and Deep Learning have been used to implement this real-time GUI-based system. Deep metric learning is the secret technique for employing deep learning for face recognition. Deep metric learning is used to solve the challenge of object recognition via deep learning. Face ROI (Regions of Interest) has been employed in this paper to detect the liveness of faces. Face ROI segmentation is used to collect facial data from a human facial image's face, neck, and lips. At First, it divides the face region horizontally into two parts (Iris and mouth), then divides the upper face region vertically into two parts (left and right eyes ROI). The basic concept of collection ROIs from faces is showed in Fig. 1. To begin, the face has been clipped

from the head area. The ROIs are then taken from the pairs of eyes, nostrils, and mouth area. Two additional libraries *dlib* [3] and *face-recognition* have been used with TensorFlow, Keras. Below is a list of the important contributions

- 1) For detecting liveness, face ROI was used with binary classification.
- 2) With our own dataset, we created a liveness detection model.
- 3) Instead of using a template-based approach, a geometry feature-based technique was used for face recognition. Face detection has been done using the HOG approach.

The rest of this paper is described as follows: Section II describes the related works applying various methods. Section III describes the approach applied in this proposed system. Section IV describes the Graphical User Interface of this system. Section V shows the discussion and the conclusion of this paper is in section VI.

II. LITERATURE REVIEW

In this proposed system, the concentration was on implementing a real-time GUI-based face recognition and liveness detection system that can accurately detect face spoofing before recognizing the faces. But there are a lot of methods and APIs to implement such types of systems. With the variety of methods, the final accuracy and execution time can differ. In this section, some different and famous methods have been described.

Different methods have been used in face recognition in different lighting, illuminations and facial expressions [4]. Open-source platform - OpenCV has been used for many years for facial experiments. OpenCV was used in an experiment on Darwin's feelings for facial expression. But before recognizing faces it is very important to detect fakeness. Because systems like face recognition are vulnerable to different types of spoofing attacks [5-7]. In the face recognition algorithm camera does not care about fakeness or spoofing. To eliminate the vulnerability, system must employ facial liveness detection. Face spoof attacks can be done by screen photo, printed or painted photo, video and 3D model of a person or only the face of person. A survey of using different schemes as the antispoofing method has been shown in [8].

Kollreider et al. [9] used an optical flow field to detect facial organ motion information for detecting liveness. Whereas in [10] eye blinking had been used for liveness detection. But still there remains vulnerability if a video is used or photographs is being shaken. In [11-12] movement of

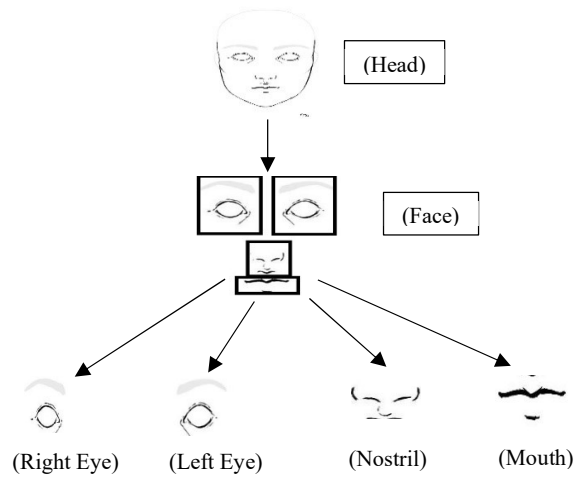


Fig. 1. The region containment tree corresponding to a face image.

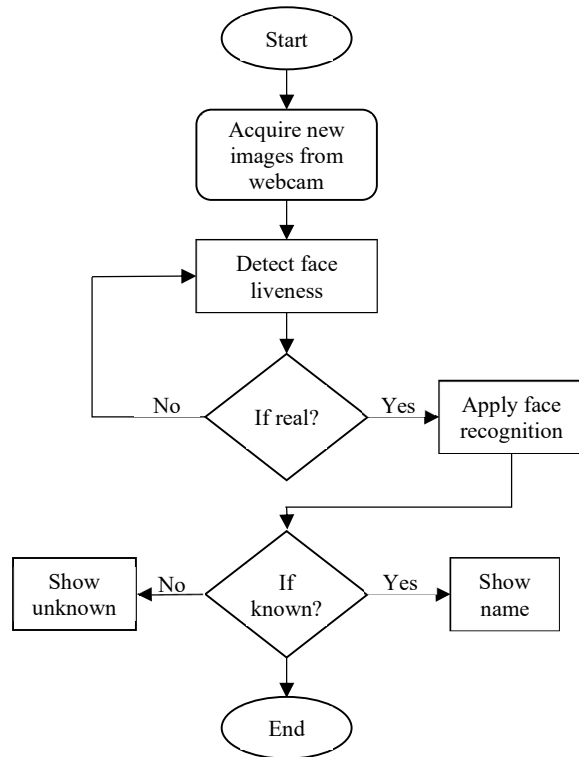


Fig. 2. Simplified flowchart of this proposed system.

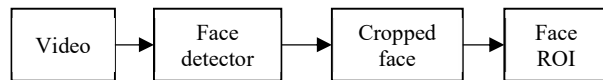


Fig. 3. Generating face ROI as training data for liveness detection.

mouth and in [13] head movement was proposed to detect face liveness. In [14] the combination of eye and mouth movement had been used to generate challenges to detect spoofing. Some researchers used smoothness, roughness and edges to detect fakeness [15]. Gahyun Kim et al used Frequency and Texture based analysis [16]. Variable Focusing-based analysis was used for liveness detection by Kim et al. [17]. But with the help of artificial intelligence or the high definition videos, this

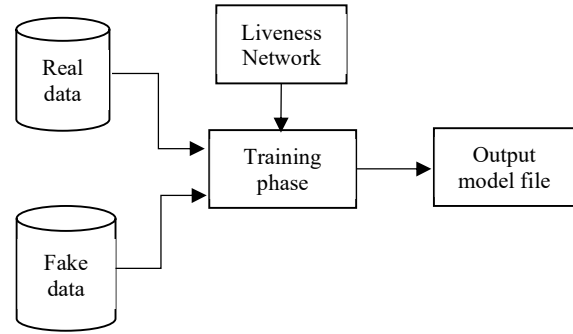


Fig. 4. Process of training liveness detector network.

methods can be hacked. Hence in this proposed system, face ROI of user has been used to detect face liveness and binary classification has been used for detecting real or fake. The proposed algorithm for using face ROI has been described in section- III.

Another part of this proposed system is the face recognition part. This part has also been a famous research topic for a very long time. Markov model, Fisher face algorithms were the common algorithms for recognition. Researchers showed the actual methodology for the recognition system in [18]. SVM (Support Vector Machine) has also been used in recognition systems [19]. In a recent publication, it has been shown that it is better to use OpenCV in IoT devices [20]. Also, Open Face which was developed by Carnegie Mellon University by OpenCV has been used as the recognition tool [21].

In this paper, a geometry feature-based algorithm has been used instead of an image template-based algorithm. That means geometric and appearance properties have been used after extracting facial features. For this purpose, *dlib* and *face recognition* package has been used with OpenCV. The *HOG* (*Histogram of Oriented Gradients*) approach has been employed to detect faces since *Haar Cascade* has more false-positives than *HOG* and *HOG* performs better in IoT devices [22].

III. METHODS

The main objectives of this proposed security system are detecting whether there is any fakeness/spoofing or not and if not then applying face recognition.

The simplified approach is showed in Fig. 2. Before implementing facial recognition, the suggested method identifies the presence of face liveness. Face recognition does not need to be applied if there is no liveness to the face.

The whole approach for this proposed system is combination of the four following steps:

- Implementing Liveness Detector
- Implementing Face Recognizer
- Using Liveness Detector
- Using Face Recognizer

A. Implementing Liveness Detector

To implement the liveness detector, the following pipeline has been followed: First and foremost, the necessary dataset was prepared. Then a CNN (Convolutional Neural Network)

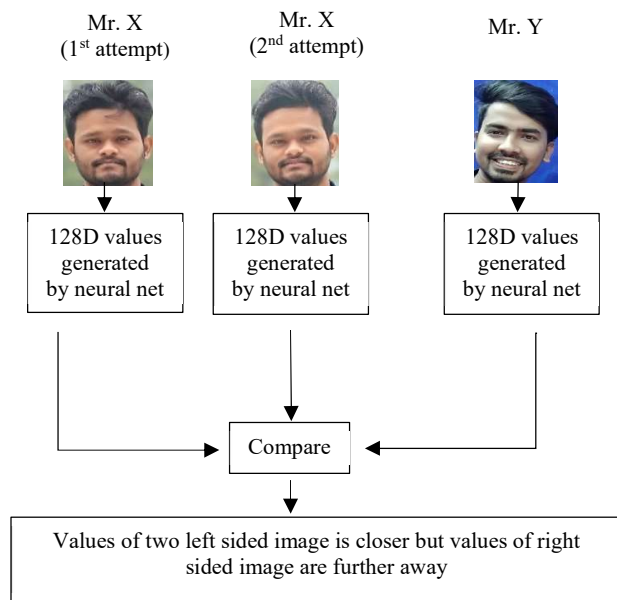


Fig. 5. Single triplet training step.

has been implemented before training the network. As previously stated, liveness detection has been done as a binary classification problem. Two types of data have been used. One acts as “real” and the other acts as “fake”. This binary categorized data was created by these following steps-

- A roughly 30 seconds video of single face was recorded from cell phone (it acts as “real” for the proposed system)
- This roughly 30 seconds video was played on desktop and then re-recorded it from the same cell phone (it acts as “fake”).
- After that, face detection has been applied to extract face ROI from both of these videos. ROI from the real video will be the “real” labeled data and another will be the “fake” labeled data.

This process has been used for 3 different times with 3 different faces for detecting different skin tones so that the system can show high accuracy for different skin. For filtering weak face detection 50% confidence has been used. To avoid neighboring frames, after taking every frame from “real” videos, a single frame has been skipped, while three frames have been skipped after taking every frame from “fake” videos. Deep Learning face detector of OpenCV has been used to detect faces from the videos and gathered 494 images as spoofed or fake images and 519 as real images.

Pretrained Caffe face detector has been used to locate the face ROIs. These stored ROIs have been used in CNN network that generates a model file. Fig. 3 shows the sequence of the steps of generating face ROIs.

The next step is to implement the CNN network. It is a very simple Convolutional Neural Network which has only three layers as it has been used just to define whether the input is real or spoofed. This CNN has only three channels and two classes. These two classes are - “real” and “fake/spoofed”. Here is the three layers of this CNN –

First Layer, CONV => RELU => CONV => POOL

Second Layer, CONV => RELU => CONV => POOL

And finally FC => RELU

With this CNN, the system has been trained with the real and fake data that were generated from the videos. OpenCV, Keras and deep learning have been used to train the network. Scikit-learn has been used for partitioning 75% of data for training and another 25% is for testing. A method called *fit_generator* from TensorFlow/Keras has been used for data augmentation and transforming from *predict_generator* to *predict*. After completing the training, a model extension file has been created for using liveness detection before using face recognition. For this proposed system it is *liveness.model* file. Once the model is trained and output file (*liveness.model*) is generated then the system is ready to apply liveness detection. Fig. 4 provides a clearer picture of the situation.

B. Implementing Face Recognizer

The face recognition system in this suggested system is implemented with OpenCV, Python, and Deep Learning, which includes the notion of Deep Metric Learning. Unlike deep learning, the result of deep metric learning is a real-valued feature vector. Two more libraries, *dlib* and *face_recognition*, have been used to create the face recognition system. Davis E. King is the author of the *dlib* library, which provides the deep metric learning implementation [3]. The output of the *dlib* facial recognition network is a list of 128 real-valued values. This network was trained by creator Davis E. King on ~3 million images. This *dlib* network outperforms state-of-the-art algorithms on the Labeled Faces in the Wild (LFW) dataset, with an accuracy of 99.38 percent. Adam Geitgey is the creator of *face_recognition* library. This library makes it easier to use facial recognition functionality of *dlib*. In this proposed system, the output vector of *dlib* has been used to quantify faces. This network has been trained using triplets. Fig. 5 shows that the 128-D measurement of the left sided two image of “Mr. X” will be closer as these two pictures are of the same person but the 128-D of the right sided image of “Mr. Y” will be further away as it is a different person. As it has been mentioned that the *dlib* is already trained with ~3 million data by Davis E. King. Here this pre-trained network *dlib* has been used to construct 128-D embeddings for each of the data images. These embeddings are used when the system uses the face recognition. In section D, it is discussed where K-NN (K-Nearest Neighbors) model has been used during classification and after counting the face classifier, the result of the face recognition system has been shown.

To implement face recognition, as data, 100 images of the same person have been used and named the folder as “Mr. X”. Then 128-D embeddings for each face in the dataset have been created. The same methods has been followed to obtain embeddings from each of the photos. The path of the face dataset (100 images of Mr. X) and *face_encode.pickle* file (for storing 128-D values of faces) are stored using *imutils* (package of convenience functions).

Before starting to encode the faces, the face detection method has been used. The *HOG* method has been used as described earlier. *HOG* reduces the dimensionality of face image and it is developed for detecting frontal views of faces [17]. In [18] they conclude that they also used the *HOG* algorithm to detect faces as it gives better and faster results.

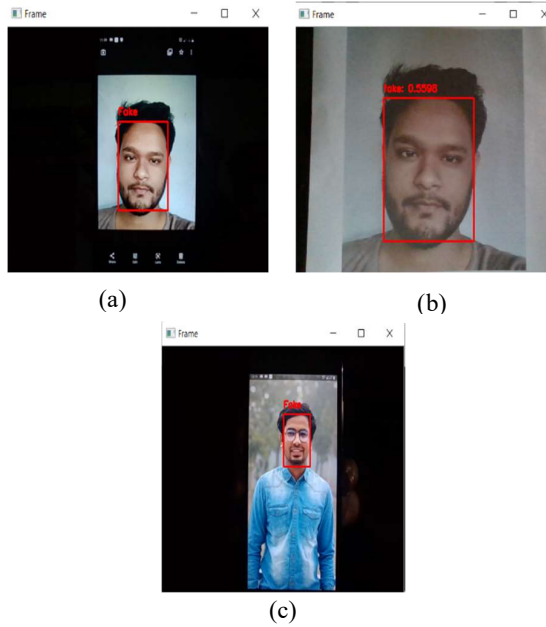


Fig. 6. Fakeness detection by the system : (a) Fake cell phone image detected for "Mr. X" (b) Fake printed image detected for "Mr. X" (c) Fake cell phone image detected for "Mr. Y".

OpenCV works in BGR (Blue, Green, Red) and *dlib* works in RGB (Red, Green, Blue). So it needs to be swapped. This step has been done by *face_recognition.face_locations()* that takes the RGB image, *HOG* method and detects face from the image. Then the *face_recognition.face_encodings()* method encodes the face into a 128-D vector. The encoded value has been stored in a list with an appropriate name (for this proposed system, folder name has been used for the name of the encoded value). As described, this encoded value with the name of each image is stored in a file named *face_encode.pickle* file. This *face_encode.pickle* file has been used at the time of using face recognition.

C. Using Liveness Detector

In section A, the *liveness.model* file was generated and placed in local disk after finishing training with OpenCV, Keras, and deep learning. The CNN training procedure is depicted in Fig. 4. The output of this training session is a model file (*liveness.model*). To use liveness detection, the *liveness.model* file needs to be used. The working pipeline of this section looks as follows-

- Accessing into webcam
- Applying face detection to each frame
- If face detected, then applying our liveness detector to detect if there is any spoofing

After accessing the webcam face needs to be detected for applying liveness detection. As a face detector, deep learning face detector of OpenCV has been used here. Using *blobFromImage* function of OpenCV, a blob has been created and then the face is detected using a face detector network.

The next step is to filter weak detection, extract the face ROI, and process it in the same way thus the liveness detector was implemented in section A. Same confidence has been used as in the section A. After that, the pre-trained serialized

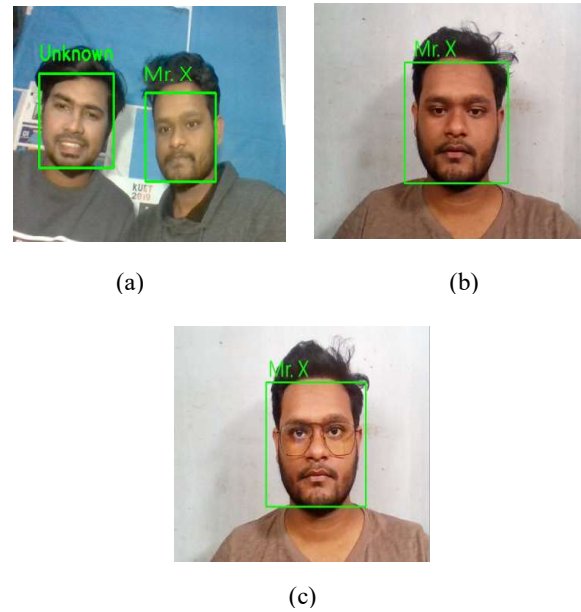


Fig. 7. Face recognition output : (a) Multiple face detected and known person recognized (b) Known person detected in different face expression and lighting (c) Known person detected with short bread and glasses.

Keras *liveness.model* file has been used to check the ROI and detected if the frame is real or not.

In Fig. 6(a) and Fig. 6(c), the system has detected the faces as fake because it is a still image on a cell phone. In Fig. 6(b) the fakeness is detected as it is a printed photo pretending to be a real user. So, according to the proposed flowchart in Fig. 2, the system should not use face recognition process. But if the system detects a real frame, then it uses face recognition to recognize the person as the system has done in Fig. 7.

D. Using Face Recognizer

In this proposed system, to use face recognition, the three required items are: the *face_encode.pickle* file created during the implementation of the face recognizer in section B, the input image from webcam, and the face detection algorithm.

The precomputed encoded file *face_encode.pickle* needs to be loaded where the name and 128-D values of each image is stored. This file has been used in this system to compare with every input image. To get input the webcam has been used and faces has been detected from the input images. After that the image has been converted into RGB color as it has been done at the time of generating the *face_encode.pickle* file in section B. The 128-D values are then computed in the same manner as before.

The final step is to compare the input 128-D values with each encoding dataset. This comparison has been done by *face_recognition.compare_face()* method. This method returns a list having data only true or false. To return these boolean values, *face_recognition.compare_face()* computes Euclidean distance between the input/candidate 128-D values and all 128-D values from the dataset. After this, the vote-counting step has been applied. The system counts the votes (the true boolean values after the comparison) for each name and then select the name with the most votes.

In Fig. 7(a) the system detects two faces and computes their 128-D values. After that, it compares their 128-D values

with the encodings data to find the match. The encodings data set has only data for “Mr. X”. As a result, it is showing “Mr. X” as a known person and the other one is “Unknown”. The system can also recognize a known person in different facial expression in Fig. 7(b) and also with glasses, a short beard in Fig. 7(c). Despite the fact that the proposed system has not been trained to recognize known persons with beards and spectacles, it is capable of doing so.

IV. GRAPHICAL USER INTERFACE

This proposed system was built as a security system based on face recognition. To assure simplicity, a GUI was created using pyqt5 designer. The following are some of the most important components of our facial recognition security system.

Fig. 8 demonstrates the graphical user interface of the proposed system. The system starts with a home page. Users can go to the other windows by clicking menu options or pressing short keys. There is a password protected admin window from where admin can add new person or delete existing person and also retrain the system. The interesting fact is that the system do not need to be retrained after renaming a person or removing a person. The trick is handling *face_encode.pickle* file. With the help of Python Pandas a name can be searched in *face_encode.pickle* file and change the name and admin can also delete the name and encodings corresponding to the name for removing any person from encodings. Thus, to rename and remove a person, retraining is not needed. Another major window is the security panel from where liveness detection starts before applying face recognition. With the press of “Start” Button the liveness detection starts to work. If the system detects fakeness then it stops applying face recognition like Fig. 6. But if there is no spoofing in front of webcam then the face recognition starts like Fig. 7.

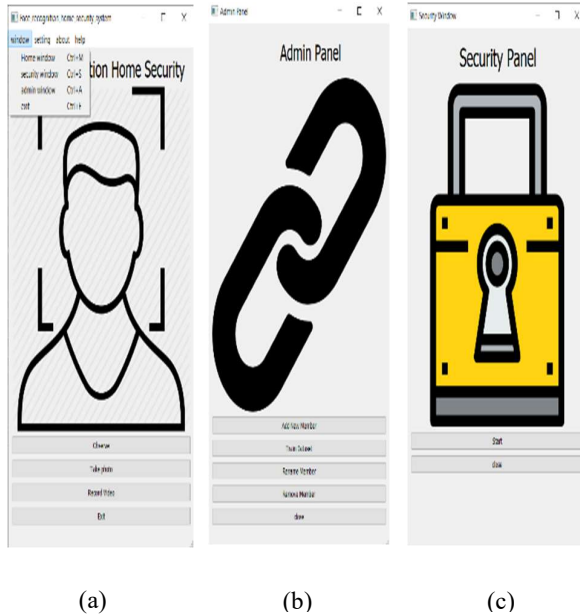


Fig. 8. Graphical user interface : (a) Home page (b) Admin Panel (c) Security panel.

TABLE I. SUMMARIZED REPORT OF SYSTEM UNDERGOING TESTING

SL No	Input Type	Output Status
1.	Photo imposter attack	Detected as “Fake”
2.	Video imposter attack	Detected as “Fake”
3.	Eye and mouth imposter attack	Detected as “Fake”
4.	Real person but not stored in dataset	Detected as “Unknown”
5.	Real known person with different facial expression and lighting	Recognized with name
6.	Real known person with different pose and scaling factor	Recognized with name
7.	Real known person with or without beard, mustache and spectacles	Recognized with name

TABLE II. EXPERIMENTAL RESULT

	No of Test (T)	No of Detection (D)	Accuracy ((D/T) %)	Average Accuracy (%)
Liveness (L)	50	49	98	$(L+K+U)/3 = 99.33$
Known person (K)	50	50	100	
Unknown person (U)	50	50	100	

V. DISCUSSION

In short, this proposed system is a combination of face liveness detection and face recognition. To implement liveness detection a dataset of 494 fake images and 519 real images has been used. After training a CNN network, the liveness detection has been solved as a binary classification problem where the two classes are “real” and “fake/spoofed”.

To get the result of liveness detection, ROI of images has been used. After detecting real images in real-time face recognition is applied in this GUI-based system. For recognizing faces a pretrained network- *dlib* has been used with the *face_recognition* package. The 128-D vector of the images with corresponding person names has been used as a dataset in a *face_encode.pickle* file. After converting every detected face into 128-D values, With the help of the *face_recognition.compare()* module this input has been compared with the stored encodings.

After the comparison, the system shows the result of recognition following the process as mentioned above. There are some issues that can be different in various situations. The accuracy sometimes depends on the hardware like the webcam, APIs, algorithm. The threshold value for detecting weak face detection, finding face ROI can also change the accuracy. Moreover, false positives depends on minimum threshold or arguments. These cases should be handled so carefully.

The summarized output status of this proposed system has been shown in table I. The project's goal was met because it can detect spoofing and then recognize known people while also detecting spoofing. According to table I, our system can detect photo imposter attacks, video imposter attacks, and eye and mouth imposter attacks. Furthermore, our system can recognize known people with a variety of facial expressions, as well as with or without a beard, mustache, or spectacles.

The experimental results of the suggested system are shown in Table II. We ran 50 test cases for liveness, including photo imposter, video imposter, and mouth imposter attacks. Out of 50 test situations, 49 were successfully identified. Our proposed method detected known and unknown people in a variety of face expressions, lighting conditions, and facial poses. As a result, the ultimate accuracy was 99.33 percent.

VI. CONCLUSION

Face recognition-based security systems had long been a research topic. As a result, there are many open source modules, libraries, and packages available and the field of computer vision is rapidly developing. All the modules, packages used in this proposed system are open source and simple to use. Pyqt5 designer has been linked with a graphical user interface to create it user friendly. This type of technology can be enhanced with additional features such as distinguishing twin faces and gender differentiation after detecting faces. Detecting and recognizing so many faces in a crowd is likewise a difficult task. As there are so many methods to implement face recognition and liveness detection, in the future this type of systems will be used not only for fun or academic projects but also in solving different types of real-time problems.

REFERENCES

- [1] Jain, A., Kumar, A.: *Biometrics of Next Generation: an Overview, Second Generation Biometrics*. Springer, Heidelberg (2010).
- [2] A.F. Abate, M. Nappi, D. Riccio, and G. Sabatino, "2D and 3D face recognition: A survey", *Pattern Recognition Letters*, vol.28, issue 15, pp.1885-1906, Oct 2007.
- [3] King, Davis E. "Dlib-ml: A machine learning toolkit." *The Journal of Machine Learning Research* 10 (2009): 1755-1758.
- [4] w. Zhao, R. Chellappa, A. Rosenfeld, P.J. Phillips, *Face Recognition: A Literature Survey*, *ACM Computing Surveys*, 2003, pp. 399-458.
- [5] Biggio, B.; Akhtar, Z.; Fumera, G.; Marcialis, G.L.; Roli, F.;, "Security evaluation of biometric authentication systems under real spoofing attacks," *Biometricals, IET*, vol.1, no. 1, pp.11-24, March 2012.
- [6] K. Nixon, V. Aimale, and R. Rowe, "Spoof detection schemes," in *Handbook of Biometrics*. Springer US, 2008.
- [7] S. A. C. Schuckers, "Spoofing and Anti-Spoofing Measures," vol. 7, *Information Security Tech. Rep.*, 2002.
- [8] J. Galbally, S. Marcel, J. Fierrez, "Biometric Antispoofing Methods: A Survey in Face Recognition", *IEEE Access*, 2015, pp. 1530-1552.
- [9] K. Kollreider, H. Fronthaler, J. Bigun, "Verifying liveness by multiple experts in face biometrics". *Computer Vision and Pattern Recognition Workshops, CVPRW 2008*, IEEE, 2008.
- [10] S.G. Pan, L.Z. Wu, S. Lao, "Blinking Based Live Face Detection Using Conditional Random Fields", *ICB 2007*, Springer, pp. 252-260, 2007.
- [11] S. M. Hatture, P. R. Karchi, "Prevention of spoof attack in biometric system using liveness detection", *Int. J. Latest Trends Eng. Tech., Special Issue-IDEAS*, pp. 42-49, 2013.
- [12] C. Kant, N. Sharma, "Fake face recognition using fusion of thermal imaging and skin elasticity". *IJCSIJ*, 4(1), pp. 65-72, 2013.
- [13] A. Lagorio, M. Tistarelli, M. Cadoni, C. Fookes, B. Clinton and S. Sridha, "Liveness detection based on 3D face shape analysis", *Proceedings of the IEEE International Workshop on Biometrics and Forensics (IWBF)*, pp. 1-4, 2013.
- [14] Singh, Avinash Kumar, Piyush Joshi, and Gora Chand Nandi. "Face recognition with liveness detection using eye and mouth movement." 2014 international conference on signal propagation and computer technology (ICSPCT 2014). IEEE, 2014.
- [15] Peixoto, B.; Michelassi, c.; Rocha, A.;, "Face liveness detection under bad illumination conditions," *Image Processing (Ic/P)*, 2011 18th IEEE International Conference on , vol., no., pp.3557-3560, 11-14 Sept. 2011.
- [16] G. Kim, S.Eum, J. K. Suhr, D. I. Kim, K. R. Park, and J. Kim, Face liveness detection based on texture and frequency analyses, 5th IAPR International Conference on Biometrics (ICB), New Delhi, India. pp. 67-72, March 2012.
- [17] sooyeon Kim, Sunjin Yu, Kwangtaek Kim, Yuseok Ban, Sangyoun Lee, Face liveness detection using variable focusing, *Biometrics (ICB)*, 2013 International Conference on, On page(s): 1-6, 2013.
- [18] Z. Rybchak, and O. Basystiuk, "Analysis of computer vision and image analysis technics," *ECONTECHMOD: an international quarterly journal on economics of technology and modelling processes*, Lublin: Polish Academy of Sciences, vol. 6, no. 2, pp. 79-84, 2017.
- [19] Dadi, Harihara Santosh, and GK Mohan Pillutla. "Improved face recognition rate using HOG features and SVM classifier." *IOSR Journal of Electronics and Communication Engineering* 11.4 (2016): 34-44.
- [20] Boyko, Nataliya, Oleg Basystiuk, and Nataliya Shakhovska. "Performance evaluation and comparison of software for face recognition, based on dlib and opencv library." 2018 IEEE Second International Conference on Data Stream Mining & Processing (DSMP). IEEE, 2018.
- [21] Prasanna, D. Mary, and Ch Ganapathy Reddy. "Development of Real Time Face Recognition System Using OpenCV." *Development* 4.12 (2017): 791.
- [22] Rahmad, C., et al. "Comparison of Viola-Jones Haar Cascade classifier and histogram of oriented gradients (HOG) for face detection." *IOP conference series: materials science and engineering*. Vol. 732. No. 1. IOP Publishing, 2020.